

WEEK-7

Task-1: Write a program that prints the addresses Of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Code:

```
root@DESKTOP-QU0HVJI: ~/t  ×  +  ▾
GNU nano 7.2                                     address.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;

void code_function()
{
    printf("Code address  : %p\n", (void *)code_function);
}

int main()
{
    static int static_var = 20;
    int stack_var = 30;
    int *heap_var = malloc(sizeof(int));

    if (heap_var == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    *heap_var = 40;

    printf("Memory Addresses:\n");

    code_function();

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address  : %p\n", (void *)heap_var);
    printf("Stack address  : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

Output:

```
root@DESKTOP-QU0HVJI:~/practical# cd practical-7/
root@DESKTOP-QU0HVJI:~/practical/practical-7# nano address.c
root@DESKTOP-QU0HVJI:~/practical/practical-7# gcc address.c -o address
root@DESKTOP-QU0HVJI:~/practical/practical-7# ./address
Memory Addresses:
Code address   : 0x56cb0669b1c9
Global address : 0x56cb0669e010
Static address : 0x56cb0669e014
Heap address   : 0x56cb3129e2a0
Stack address  : 0x7fff33cdf56c
root@DESKTOP-QU0HVJI:~/practical/practical-7# |
```

Report: The program was successfully implemented to display addresses of different memory segments.

Addresses of code, global, static, heap, and stack variables were observed.

The results showed that different types of data are stored in different regions of virtual memory. Linux uses virtual address space and address randomization, so the exact addresses may change between executions.

Task-2: Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Code:

```
GNU nano 7.2 maps.c *
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Process ID: %d\n", getpid());
    printf("Process is running...\n");
    printf("Check /proc/%d/maps from another terminal.\n", getpid());

    while (1)
    {
        sleep(5);
    }

    return 0;
}
```

Output:

```
root@DESKTOP-QU0HVJI:~# /proc/432/maps
-bash: /proc/432/maps: Permission denied
root@DESKTOP-QU0HVJI:~# cat /proc/432/maps
653072937000-653072938000 r--p 00000000 08:30 46447 /root/pratical/practical-7/maps
653072938000-653072939000 r-xp 00001000 08:30 46447 /root/pratical/practical-7/maps
653072939000-65307293a000 r--p 00002000 08:30 46447 /root/pratical/practical-7/maps
65307293a000-65307293b000 r--p 00002000 08:30 46447 /root/pratical/practical-7/maps
65307293b000-65307293c000 rw-p 00003000 08:30 46447 /root/pratical/practical-7/maps
65309ac7f000-65309aca0000 rw-p 00000000 00:00 0 [heap]
7a4d6e600000-7a4d6e628000 r--p 00000000 08:30 49944 /usr/lib/x86_64-linux-gnu/libc.so.6
7a4d6e628000-7a4d6e7b0000 r-xp 00028000 08:30 49944 /usr/lib/x86_64-linux-gnu/libc.so.6
7a4d6e7b0000-7a4d6e7ff000 r--p 001b0000 08:30 49944 /usr/lib/x86_64-linux-gnu/libc.so.6
7a4d6e7ff000-7a4d6e803000 r--p 001fe000 08:30 49944 /usr/lib/x86_64-linux-gnu/libc.so.6
7a4d6e803000-7a4d6e805000 rw-p 00202000 08:30 49944 /usr/lib/x86_64-linux-gnu/libc.so.6
7a4d6e805000-7a4d6e812000 rw-p 00000000 00:00 0
7a4d6e9ec000-7a4d6e9ef000 rw-p 00000000 00:00 0
7a4d6e9f4000-7a4d6e9f6000 rw-p 00000000 00:00 0
7a4d6e9f6000-7a4d6e9fa000 r--p 00000000 00:00 0 [vvar]
7a4d6e9fa000-7a4d6e9fc000 r--p 00000000 00:00 0 [vvar_vclock]
7a4d6e9fc000-7a4d6e9fe000 r-xp 00000000 00:00 0 [vdso]
7a4d6e9fe000-7a4d6e9ff000 r--p 00000000 08:30 49941 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7a4d6e9ff000-7a4d6ea2a000 r-xp 00001000 08:30 49941 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7a4d6ea2a000-7a4d6ea34000 r--p 0002c000 08:30 49941 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7a4d6ea34000-7a4d6ea36000 r--p 00036000 08:30 49941 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7a4d6ea36000-7a4d6ea38000 rw-p 00038000 08:30 49941 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fff5256b000-7fff5258c000 rw-p 00000000 00:00 0 [stack]
root@DESKTOP-QU0HVJI:~#
```

Report: The `/proc/PID/maps` file was used to study the virtual memory layout of a running process.

Different mappings such as code, data, heap, stack, and shared libraries were observed.

The permissions of each region were analyzed using read, write, and execute flags.

This experiment showed how Linux organizes and manages a process using virtual memory mappings.